

Computational Complexity: Classes

AA-GEI FIB, UPC

Spring 2025-2026

- Les classes permettent de dire que je ne peux pas faire, parce que c'est un problème d'algorithme polynômial temps.

Problem types

- **Decision** (Si o no)
 Input x in I
 Is $P(x)$ true?

Example: Given a graph and two vertices, is there a path joining them?

- **Function** (retorna 2 valores) (Si Salida es exp $\Rightarrow$ algo exp $\wedge$ B)
 Input x in I
 Compute y such that $Q(x, y)$

Example: Given a graph and two vertices, compute the minimum distance between them.

Problem types

- Decision

Input x

Property $P(x)$

$\langle x \rangle \equiv$ entrada codificada

$(0, 1)$
 $\uparrow$

By coding inputs on alphabet Σ such a problem identifies a language:

$$L = \{ \langle x \rangle \in \Sigma^* \mid P(x) \} \in \mathcal{P}(\Sigma^*) \quad (\text{entrada que pertenece a } (0,1)^* : \text{retornar si})$$

- Function

Input x

Compute y such that $Q(x, y)$

By coding inputs/outputs on alphabet Σ a deterministic algorithm solving a problem determines a partial function

$f : \Sigma^* \rightarrow \Sigma^*$ s.t., for any x , $Q(x, f(x))$ is true.

poner de los vertices a un cono!

- We need to guarantee that $\{ \langle x \rangle \mid x \in I \}$ is decidable. $\left(\begin{array}{l} \text{inputs han de ser} \\ \text{decible!} \end{array} \right)$

$(I \equiv \text{input})$

Decision problem classes

- **Undecidable**
No algorithm can solve the problem.
- **Decidable**
There is an algorithm solving them.

Classifying decidable problems

- Decidable problems, admit algorithmic solutions.
- The algorithms solving such problems can use more or less amount of resources.
- To measure the performance of an algorithm we use two measures **time** and **space**.
- As usual we take a worst case analysis on inputs with the same size.
- For a TM, time correspond to the **length of the computation** and space to the portion of the tape **accessed** during the computation.
- Observe that sometimes the space used might be smaller than the input.

Complexity classes: definitions

$f(n)$ pot ser $n^2, n^3, n^4, \dots$ o n

Let $f : \mathbb{N} \rightarrow \mathbb{N}$,

- The class **TIME**($f(n)$) is the class of decision problems for which an algorithm exists that solves instances of size n in time $O(f(n))$.
- The class **SPACE**($f(n)$) is the class of decision problems for which an algorithm exists that solves instances of size n using space $O(f(n))$.

Some complexity classes

- $P = \cup_k TIME(n^k) = TIME(poly(n))$
- $EXP = TIME(2^{poly(n)})$
- $2EXP = TIME(2^{2^{poly(n)}})$
- $L = LOGSPACE = SPACE(\lg n)$
- $PSPACE = SPACE(poly(n))$
- $EXSPACE = SPACE(2^{poly(n)})$

$$L \subseteq PSPACE \subseteq EXSPACE$$

$$L \subseteq P \subseteq EXP \quad PSPACE \subseteq EXP$$

Non deterministic machines

↳ La màquina decideix que val fer, ningú més, no sap que farà!

- We can use a generalized, although unrealistic, model of computation.
- A program has a built-in operation allowing to branch to different positions in the code.
- When the code is executed, the machine follows the the “best” continuation.

(un 'best' encull sempre
la millor opció)

Complexity classes: definitions

Let $f : \mathbb{N} \rightarrow \mathbb{N}$,

- The class $\text{NTIME}(f(n))$ is the class of decision problems that a non deterministic TM recognizes in time $O(f(n))$.
- The class $\text{NSPACE}(f(n))$ is the class of decision problems for which a non deterministic TM solves instances of size n using space $O(f(n))$.

Some complexity classes

- $NP = NTIME(poly(n))$
- $NEXP = NTIME(2^{poly(n)})$
- $NL = NSPACE(\lg n)$
- $NPSPACE = NSPACE(poly(n)) \rightarrow \text{el millon bits en el bit pel 'llevit'}$

$$L \subseteq NL \subseteq P$$

$$P \subseteq NP \subseteq PSPACE = NPSPACE \subseteq EXP$$

Logic and NP

- A **verifier** for a language L is an algorithm $\mathcal{A}$, where $L = \{x \mid \mathcal{A} \text{ accepts } \langle x, c \rangle \text{ for some word } c\}$.
- A **polynomial time verifier** runs in polynomial time in the length of x .
- A language L is **polynomially verifiable** if it has a polynomial time verifier.
- A verifier uses additional information, represented by the word c , to verify that a string x is a member of L . This information is called a **certificate**, or **proof**, of membership in L .
- A polynomial verifier can access only polynomial space. Therefore, we can assume that $|c|$ is polynomial in $|x|$.

Decision Problems in NP: Examples

result
 Rechenfunktion - kann auf result
 dem result

$L \in NP$ iff $L = \{x \mid \exists y R(x, y)\}$ where $|y| = \text{poly}(|x|)$ and R can be decided in polynomial time.

NP is the class of polynomially verifiable languages.

Decision Problems in NP: Examples

- **COMPOSITE:** Given an integer n , is n composite? $x = n$
 $y = n_1 \wedge n_2$
 $R = n_1 x \wedge n_2 = n$

 - $\text{COMPOSITE} = \{n \mid \exists n_1, n_2 \ n = n_1 \times n_2\}$
 - The certificate has polynomial size, and the verifier runs in polynomial time.
 - So, $\text{COMPOSITE} \in \text{NP}$.
- **SUBSET-SUM:** Given a set $S = \{x_1, \dots, x_n\}$ of $\mathbb{Z}^+$ and $t \in \mathbb{Z}$, exists $S' \subseteq S$ with $\sum_{x_j \in S'} x_j = t$?

 - The certificate is a subset of S , it can be represented by a boolean vector, so it has polynomial size. The verifier runs in polynomial time.
 - So, $\text{SUBSET-SUM} \in \text{NP}$.
- **HAMILTONIAN CYCLE:** Given a graph G , is G Hamiltonian?

 - The certificate is a permutation of $V(G)$, it can be represented by a vector, so it has polynomial size. The verifier also runs in polynomial time.
 - So, $\text{HAMILTONIAN CYCLE} \in \text{NP}$.

Classes of complements (Complementaire)

- Let $\mathcal{C}$ be a class of decision problems. $\text{co-}\mathcal{C}$ is formed by the complements of languages in $\mathcal{C}$, i.e., $L \in \text{co-}\mathcal{C}$ iff $\bar{L} \in \mathcal{C}$.
- Classes defined by TMs are closed under complement.

$$P = \text{co-}P \quad \text{EXP} = \text{co-EXP} \quad \text{PSPACE} = \text{co-PSPACE}$$

- Not so clear for classes defined by NTMs.
 $\text{co-NP} = \{x \mid \forall y R(x, y)\}$ where $|y| = \text{poly}(|x|)$ and R can be decided in polynomial time.
- It seems different to assess that a property holds for a leaf in a tree than on all their leaves.

Decision Problems in co-NP: Examples

Co-NP is a good example / prove with TQTS!

- **PRIME**: Given an integer n , is n prime?
 - $\text{PRIME} = \{n \mid \forall a \leq n \ n \bmod a \neq 0\}$.
 - PRIME is the complement of COMPOSITE , we have $\text{PRIME} \in \text{co-NP}$
 - $\text{PRIME} \in \text{NP}$ as it can be verified in poly time (Pratt 1975). So in $(\exists y) \text{NP} \cap \text{co-NP} \sim (\forall y)$
 - A polynomial time algorithm $O((\lg n)^6)$ was devised by Agrawal, Kayal and Saxena in 2002.

Decision Problems in NP: Examples

- **FACTORIZATION:** Given an integer n , are there primes $p_1, \dots, p_k$ s.t. $x = p_1 \times \dots \times p_k$.
 - $p_1, \dots, p_k$ is the certificate.
 - k and all the possible factors p are $\leq n$.
 - The verifier, on input n and $p_1, \dots, p_k$, computes $p_1 \times \dots \times p_k$, checks if the result is n . If so, checks that all the values are prime numbers.
 - This takes time $O(k \lg n^2 + k \lg n^6)$.
 - So, FACTORIZATION $\in$ NP.

Open questions

- Is $P = NP$?
- $P \neq EXP$, but is $NP = EXP$?
- $P \subseteq NP \cap co-NP$, but is $P = NP \cap co-NP$?
- Is $L = NL$?

Efficient algorithms?

Feasible problem there is an algorithm to find a solution **efficiently**.

The Determinant Problem: Given an $n \times n$ matrix $M = (m_{ij})$, compute

$$\det(M) = \sum_{\pi \in S_n} (-1)^\pi \prod_{i=1}^n m_{i,\pi(i)},$$

where $(-1)^\pi = \begin{cases} -1 & \text{if } \pi \text{ has odd parity,} \\ +1 & \text{if } \pi \text{ has even parity.} \end{cases}$

Can be solved in $O(n^3)$, using the LU decomposition.

Efficient algorithms?

The Permanent Problem: Given an $n \times n$ matrix $M = (m_{ij})$, compute

$$\text{perm}(M) = \sum_{\pi \in S_n} \prod_{i=1}^n m_{i,\pi(i)}.$$

$$\text{Perm} \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = aei + bfg + cdh + ceg + bdi + afh$$

Probably exponential **Valiant 1979**, best complexity known, $O(n2^n)$. **Glynn 2010**

Main difference $\det(M_1 \times M_2) = \det(M_1) \times \det(M_2)$ but $\text{perm}(M_1 \times M_2) \neq \text{perm}(M_1) \times \text{perm}(M_2)$

Efficient algorithms?

- In computational complexity efficient algorithm is synonym of polynomial time.
- In practice, it is synonym of a low degree polynomial time.
- Nevertheless, we some times use efficient as synonym of **best known** cost.
- How to differentiate strict exponential time from polynomial time?
- We cannot do that in most of the cases, **we relate such answers to some of the open complexity questions.**

Complexity classes for other types of problems

In the same way we can define complexity classes for other types of problems

- **Function problems**

Given x , compute y such that $R(x, y)$ holds.

Classes: FP, FNP, FEXP, ...

- **Optimization problems**

Given x , compute y such that $R(x, y)$ holds and $m(x, y)$ is maximum/minimum.

Classes: PO, NPO, EXPO, ...

- **Counting problems**

Given x , how many y are there such that $R(x, y)$?

Classes: #P

Function Problems: Examples

- **Reachability:** Given a directed graph G and two vertices u and v , find a path from u to v , if one exists.
- **Eulerian cycle:** Given a graph G , find an Eulerian cycle that traverses all the edges exactly once, if such a cycle exists.
- **Factoring:** Given an integer n , find its prime factors.
 ↗ Decision $P \Rightarrow$ Search
 ↘ Func NP
- **Subset Sum:** Given a sequence of positive integers $S = \{a_1, \dots, a_n\}$ and $t \in \mathbb{Z}$, obtain $I \subseteq \{1, \dots, n\}$ s.t. $\sum_{i \in I} a_i = t$.

Optimization Problems: Examples

NP-Digraph

- **Max Cut:** Given a graph G , find a partition a partition V into V_1, V_2 , such that it maximizes the number of crossing edges between V_1 and V_2 .
- **Min Cut:** Given a digraph G , find a partition V into V_1, V_2 , such that it minimizes the number of edges going from V_1 to V_2 .
- **Shortest path:** Given a connected and edge weighted digraph G and $s, t \in V(G)$, find a path between s and t minimizing the sum of the weights in the path.

↓
(flow)

Counting problems

- How many perfect matchings are there for a given bipartite graph?
- How many graph colorings using k colors are there for a particular graph G ?
- What is the value of the permanent of a given matrix whose entries are 0 or 1?
- How many different variable assignments will satisfy a given general boolean formula?

More complexity classes?

Sure, plenty!

Look into the [Complexity zoo](#).

547 classes the last time I've had a look!

